



UFAM

UNIVERSIDADE FEDERAL DO AMAZONAS – UFAM
FACULDADE DE ENGENHARIA ELÉTRICA E DE COMPUTAÇÃO –
FEEC

LARISSA RAFAELA RIBEIRO DE SOUZA
SAMY YANO MALLMANN

PROCESSADOR DE 8 BITS

Desenvolvimento da Unidade de Controle via Máquina de Estados Finitos e
Verificação em FPGA

MANAUS – AM

2026

LARISSA RAFAELA RIBEIRO DE SOUZA
SAMY YANO MALLMANN

PROCESSADOR DE 8 BITS

Desenvolvimento da Unidade de Controle via Máquina de Estados Finitos e
Verificação em FPGA

Relatório técnico apresentado como requisito parcial
para obtenção da aprovação na disciplina de Eletrô-
nica Digital II e Laboratório de Eletrônica Digital mi-
nistrada pelo professor Thiago Brito na Universidade
Federal do Amazonas.

Área de concentração: Engenharia Elétrica e da
Computação

Orientador: Prof. Dr. Thiago Brito

MANAUS – AM

2026

Sumário

1	Introdução	5
1.1	Objetivos	6
1.1.1	Objetivo Geral	6
1.1.2	Objetivos Específicos	6
1.2	Gerenciamento do Projeto	6
2	Arquitetura do Processador	7
2.1	Visão Geral e Diagrama de Blocos	7
2.2	Barramento W	8
2.3	Conjunto de Instruções	8
2.4	Palavra de Controle (12 bits)	9
3	Máquina de Estados Finitos (FSM)	11
3.1	Ring Counter e os Seis Estados (T1–T6)	11
3.1.1	Implementação no Verilog	11
3.2	Ciclo de Busca (T1, T2, T3)	12
3.2.1	T1 — Estado de Endereço	12
3.2.2	T2 — Estado de Incremento	12
3.2.3	T3 — Estado de Memória	13
3.3	Ciclo de Execução (T4, T5, T6)	13
3.3.1	Instrução LDA (Load Accumulator)	14
3.3.2	Instrução ADD (Adição)	14
3.3.3	Instrução SUB (Subtração)	14
3.3.4	Instrução OUT (Saída)	14

3.3.5	Instrução HLT (Halt)	15
3.4	Tabela de Controle Completa	15
3.5	Diagrama da FSM	16
4	Descrição dos Módulos Verilog	18
4.1	processador_8_bits_top.v — Módulo Top-Level	18
4.1.1	Interface com a DE10-Lite	19
4.1.2	Árbitro do Barramento W	19
4.1.3	Instâncias e Conexões	19
4.2	clock_divider.v — Divisor de Clock	20
4.2.1	Interface	20
4.2.2	Funcionamento	20
4.3	program_counter.v — Contador de Programa	20
4.3.1	Interface	20
4.3.2	Funcionamento	21
4.4	mar.v — Registrador de Endereço de Memória	21
4.4.1	Interface	21
4.4.2	Funcionamento	21
4.5	ram_16x8.v — Memória RAM	21
4.5.1	Interface	22
4.5.2	Programa de Teste Pré-carregado	22
4.5.3	Funcionamento	22
4.6	instruction_register.v — Registrador de Instrução	22
4.6.1	Interface	23
4.6.2	Funcionamento	23
4.7	accumulator.v — Acumulador A	23
4.7.1	Interface	23
4.7.2	Funcionamento	23
4.8	register_b.v — Registrador B	24
4.8.1	Interface	24

4.8.2	Funcionamento	24
4.9	alu.v — Somador/Subtrator	24
4.9.1	Interface	24
4.9.2	Funcionamento	25
4.10	output_register.v — Registrador de Saída	25
4.10.1	Interface	25
4.10.2	Funcionamento	25
4.11	controller_sequencer.v — Controlador/Sequenciador	25
4.11.1	Interface	26
4.11.2	Ring Counter	26
4.11.3	Geração da Palavra de Controle	26
5	Programa de Teste	28
5.1	Descrição do Programa Carregado na RAM	28
5.2	Execução Passo a Passo (Trace Completo)	29
5.3	Resultado Esperado	30
6	Simulação e Testbench	31
6.1	Descrição dos Testbenches	31
6.1.1	tb_alu.v	31
6.1.2	tb_clock_divider.v	31
6.1.3	tb_program_counter.v	32
6.1.4	tb_registradores.v	32
6.1.5	tb_controller_sequencer.v	32
6.1.6	tb_processador_8_bits.v	32
6.2	Resultados da Simulação	33
6.2.1	ALU — tb_alu.v	33
6.2.2	Divisor de Clock — tb_clock_divider.v	33
6.2.3	Program Counter — tb_program_counter.v	34
6.2.4	Registradores — tb_registradores.v	35
6.2.5	Controlador-Sequenciador — tb_controller_sequencer.v	35

6.2.6	Processador Completo — tb_processador_8_bits.v	36
6.3	Verificação do Resultado	37
7	Implementação na FPGA DE10-Lite	38
7.1	Mapeamento de Pinos	38
7.2	Clock e Divisor de Frequência	39
7.3	LEDs e Display Binário	39
7.4	Extensão Proposta: Controle do Processador por um PC Hospedeiro	41
8	Conclusão	43
	Referências	45

Capítulo 1

Introdução

O avanço da microeletrônica ao longo das últimas décadas tornou possível a criação de processadores cada vez mais complexos e eficientes. Contudo, a compreensão dos fundamentos que regem o funcionamento interno de um processador continua sendo um desafio no ensino de sistemas digitais. Nesse contexto, o SAP-1 (*Simple As Possible 1*), proposto por Albert Paul Malvino e Jerald A. Brown [1], surge como uma ferramenta pedagógica de grande valor: um processador de 8 bits deliberadamente simples, projetado para expor de forma didática todos os elementos essenciais de uma arquitetura de computador funcional.

O SAP-1 possui um conjunto de instruções reduzido — apenas cinco operações (LDA, ADD, SUB, OUT e HLT) — e uma arquitetura de barramento único de 8 bits, denominado barramento W, pelo qual todos os módulos do sistema se comunicam. Apesar de sua simplicidade, o processador contempla os conceitos fundamentais presentes em qualquer microprocessador moderno: busca e decodificação de instruções, ciclo de execução controlado por uma unidade de controle, operações aritméticas via ULA e mecanismos de entrada e saída de dados [2].

Este trabalho documenta a implementação completa do SAP-1 em Verilog HDL, desenvolvida como parte da *Lista 4* da disciplina ministrada pelo Prof. Dr. Thiago Brito na Faculdade de Eletrônica e Computação (FEEC) da Universidade Federal do Amazonas (UFAM). O projeto foi sintetizado e verificado funcionalmente na placa de desenvolvimento Terasic DE10-Lite, baseada no FPGA *Intel/Altera MAX 10* (10M50DAF484C7G) [4].

As seções seguintes descrevem a arquitetura adotada, a máquina de estados finitos implementada, os módulos Verilog desenvolvidos, os resultados da simulação e a implementação física na FPGA.

1.1 Objetivos

1.1.1 Objetivo Geral

Implementar o processador didático SAP-1 (*Simple As Possible 1*) em linguagem de descrição de hardware Verilog HDL, sintetizando-o na placa FPGA Terasic DE10-Lite (Intel/Altera MAX 10) e verificando seu funcionamento por meio de simulação e demonstração física nos LEDs da placa.

1.1.2 Objetivos Específicos

- a) Descrever em nível RTL (*Register Transfer Level*) todos os módulos que compõem a arquitetura do SAP-1: Contador de Programa (PC), MAR, RAM 16×8 , Registrador de Instrução (IR), Acumulador A, Registrador B, ALU, Registrador de Saída, Controlador/Sequenciador e Divisor de Clock;
- b) Implementar a unidade de controle como uma Máquina de Estados Finitos (FSM) com seis estados de temporização (T1–T6), utilizando um contador circular (*ring counter*) acionado pela borda negativa do clock, conforme a arquitetura original proposta por Malvino e Brown [1];
- c) Implementar o conjunto de instruções do SAP-1 — LDA, ADD, SUB, OUT e HLT — gerando corretamente a palavra de controle de 12 bits para cada estado de temporização;
- d) Resolver o conflito de barramento causado pela ausência de suporte a drivers tristate internos no FPGA MAX 10, utilizando um árbitro de barramento baseado em multiplexador prioritário no módulo top-level;
- e) Desenvolver *testbenches* em Verilog com verificação automatizada do resultado, validando a execução dos programas de testes confirmando o valor exibido nos LEDs;
- f) Demonstrar o funcionamento do processador na placa DE10-Lite, com clock reduzido a aproximadamente 1 Hz para observação visual da execução passo a passo nos LEDs.

1.2 Gerenciamento do Projeto

O planejamento e o acompanhamento das etapas do desenvolvimento — divisão de tarefas entre os integrantes, controle de módulos pendentes, testes e revisão — foram organizados por meio de um quadro Kanban na plataforma Trello, disponível em: <https://trello.com/b/1QeD2SYr/processador-8-bits-eletronica-digital-ii>

Capítulo 2

Arquitetura do Processador

2.1 Visão Geral e Diagrama de Blocos

O SAP-1 (*Simple As Possible 1*) é um processador educacional de 8 bits cuja finalidade é ilustrar, de maneira didática, o funcionamento interno de um processador real. Apesar da sua simplicidade, o SAP-1 possui todos os componentes essenciais de uma CPU: unidade de controle, unidade lógica e aritmética, registradores, memória e interface de saída.

A Figura 2.1 apresenta o diagrama de blocos do processador implementado em Verilog, conforme instanciado no módulo `processador_8_bits_top.v`. Todos os módulos comunicam-se por meio de um barramento único de 8 bits denominado Barramento W.

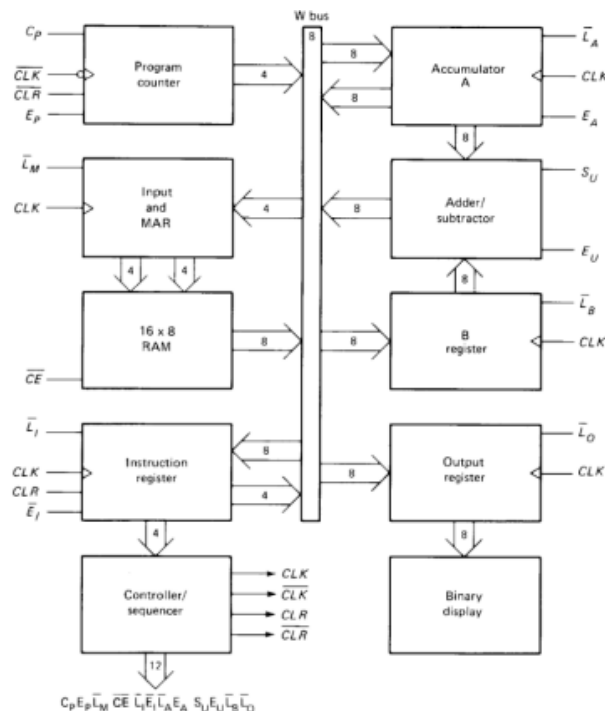


Figura 2.1: Diagrama de blocos do processador SAP-1 implementado na FPGA.

Os módulos que compõem o processador, todos instanciados em `processador_8_bits_top.v`, são:

1. **Divisor de Clock** (`clock_divider.v`)
2. **Contador de Programa** (`program_counter.v`)
3. **Registrador de Endereço de Memória — MAR** (`mar.v`)
4. **Memória RAM 16×8** (`ram_16x8.v`)
5. **Registrador de Instruções** (`instruction_register.v`)
6. **Acumulador A** (`accumulator.v`)
7. **Registrador B** (`register_b.v`)
8. **ALU — Somador/Subtrator** (`alu.v`)
9. **Registrador de Saída** (`output_register.v`)
10. **Controlador/Sequenciador** (`controller_sequencer.v`)

2.2 Barramento W

O Barramento W é o único meio de comunicação entre os módulos do `processador_8_bits`. Trata-se de um barramento de 8 bits pelo qual trafegam tanto endereços (4 bits, nas posições inferiores) quanto dados (8 bits completos). Em um hardware real, o barramento seria implementado com lógica tristate; entretanto, a família *MAX 10* da Intel (FPGA da placa DE10-Lite) não suporta tristate interno. Por isso, o projeto adota um multiplexador prioritário no módulo `processador_8_bits_top.v` para arbitrar qual módulo dirige o barramento em cada ciclo de clock.

A ordem de prioridade — PC, RAM, IR, Acumulador e ALU — garante que, a cada estado de temporização, apenas um módulo dirige o barramento. A implementação em Verilog e o detalhamento de cada sinal de habilitação são apresentados na Seção 4.1 (Capítulo 4).

2.3 Conjunto de Instruções

O processador implementa cinco instruções de máquina, cada uma codificada em 8 bits. Os 4 bits mais significativos formam o *opcode* e os 4 bits menos significativos formam o campo de endereço (operando), conforme ilustrado na Figura 2.2.

Opcode [7:4]				Endereço [3:0]			
7	6	5	4	3	2	1	0

Figura 2.2: Formato de instrução do processador: nibble alto = opcode; nibble baixo = operando.

A Tabela 2.1 descreve o conjunto de instruções implementado, com os respectivos opcodes conforme definidos em `controller_sequencer.v`.

Tabela 2.1: Conjunto de instruções do processador.

Mnemônico	Opcode (binário)	Operação	Hex
LDA	0000	Acumulador $\leftarrow$ RAM[endereço]	0
ADD	0001	Acumulador $\leftarrow$ Acumulador + RAM[endereço]	1
SUB	0010	Acumulador $\leftarrow$ Acumulador – RAM[endereço]	2
OUT	1110	Registrador de Saída $\leftarrow$ Acumulador	E
HLT	1111	Para o processamento	F

2.4 Palavra de Controle (12 bits)

O Controlador/Sequenciador emite, a cada estado de temporização, uma palavra de controle de 12 bits que determina o comportamento de cada módulo. O formato é:

$$\text{CON} = \underbrace{C_p}_{11} \underbrace{E_p}_{10} \underbrace{\overline{L_m}}_9 \underbrace{\overline{CE}}_8 \underbrace{\overline{L_i}}_7 \underbrace{\overline{E_i}}_6 \underbrace{\overline{L_a}}_5 \underbrace{E_a}_4 \underbrace{S_u}_3 \underbrace{E_u}_2 \underbrace{\overline{L_b}}_1 \underbrace{\overline{L_o}}_0$$

A Tabela 2.2 descreve cada sinal de controle, seu módulo associado e a ação realizada quando ativo.

Tabela 2.2: Sinais da palavra de controle do processador.

Sinal	Nome	Ação quando ativo	Nível ativo
C_p	Counter Program	Incrementa o PC	Alto
$\overline{E_p}$	Enable Program Counter	PC $\rightarrow$ Barramento W	Alto
$\overline{L_m}$	Load MAR	Barramento W $\rightarrow$ MAR	Baixo
$\overline{CE}$	Chip Enable RAM	RAM[MAR] $\rightarrow$ Barramento W	Baixo
$\overline{L_i}$	Load Instruction Reg.	Barramento W $\rightarrow$ IR	Baixo
$\overline{E_i}$	Enable Instruction Reg.	Operando IR $\rightarrow$ Barramento W	Baixo
$\overline{L_a}$	Load Accumulator A	Barramento W $\rightarrow$ Acumulador	Baixo
E_a	Enable Accumulator A	Acumulador $\rightarrow$ Barramento W	Alto
S_u	Subtract	Seleciona subtração na ALU	Alto
E_u	Enable ALU	ALU $\rightarrow$ Barramento W	Alto
$\overline{L_b}$	Load Register B	Barramento W $\rightarrow$ Reg. B	Baixo
$\overline{L_o}$	Load Output Register	Barramento W $\rightarrow$ Reg. Saída	Baixo

Os sinais com barra superior (*overline*) são ativos em nível baixo (0); os demais são ativos em nível alto (1). O valor inativo padrão de cada sinal é definido no bloco `always @(*)` do controlador antes de qualquer decodificação de estado.

Tabela 2.3: Tabela de controle completa do processador (ciclos de busca e execução).

Ciclo	Instrução	Estado	Sinais Ativos	Palavra de Controle
Busca	Todas	T1	$E_p, \overline{L_m}$	0101 1110 0011
		T2	C_p	1011 1110 0011
		T3	$\overline{CE}, \overline{L_i}$	0010 0110 0011
Execução	LDA	T4	$\overline{E_i}, \overline{L_m}$	0001 1010 0011
		T5	$\overline{CE}, \overline{L_a}$	0010 1100 0011
		T6	(nenhum)	0011 1110 0011
Execução	ADD	T4	$\overline{E_i}, \overline{L_m}$	0001 1010 0011
		T5	$\overline{CE}, \overline{L_b}$	0010 1110 0001
		T6	$\overline{L_a}, E_u$	0011 1100 0111
Execução	SUB	T4	$\overline{E_i}, \overline{L_m}$	0001 1010 0011
		T5	$\overline{CE}, \overline{L_b}$	0010 1110 0001
		T6	$\overline{L_a}, E_u, S_u$	0011 1100 1111
Execução	OUT	T4	$E_a, \overline{L_o}$	0011 1111 0010
		T5	(nenhum)	0011 1110 0011
		T6	(nenhum)	0011 1110 0011
Execução	HLT	T4	$hlt = 1$ (congela o ring)	---
		T5	hlt mantido	---
		T6	hlt mantido	---

A Tabela 2.3 apresenta a tabela completa de controle para todos os estados (T1–T6) e todas as instruções, conforme especificado na Lista 4 e implementado em `controller_sequencer.v`.

Capítulo 3

Máquina de Estados Finitos (FSM)

O controlador-sequenciador do SAP-1 é o coração do processador: é ele quem determina, a cada ciclo de clock, quais registradores devem operar e em qual direção os dados fluem pelo Barramento W. Para exercer esse controle de forma ordenada, o módulo implementa uma Máquina de Estados Finitos (FSM) composta por seis estados de temporização, denominados T1 a T6, organizados em um *ring counter* (contador circular).

3.1 Ring Counter e os Seis Estados (T1–T6)

Conforme especificado na Lista 4, o sequenciamento dos estados é realizado por um contador circular de 6 bits, onde apenas um bit permanece em nível alto por vez. Cada bit representa um estado de temporização: ring[0] = T1, ring[1] = T2, ..., ring[5] = T6.

3.1.1 Implementação no Verilog

O *ring counter* é sensível à borda negativa do clock (conforme a Lista 4) e ao reset assíncrono ativo em baixo (rst_n). Após o reset, o processador parte do estado T1 (ring = 6'b000001). A rotação é implementada por um deslocamento circular simples:

```

1 always @(negedge clk or negedge rst_n) begin
2     if (!rst_n)
3         ring <= 6'b000001;          // inicia em T1
4     else if (!hlt)
5         ring <= {ring[4:0], ring[5]}; // T1->T2->T3->T4->T5->T6->T1
6     % quando hlt=1: ring não avança - processador congela
7 end

```

Listing 3.1: Contador em anel no módulo controller_sequencer.v.

Quando a instrução HLT é detectada no estado T4, o sinal `hlt` é ativado em nível alto e o *ring counter* deixa de avançar, paralisando o processador sem necessidade de interromper o clock global — estratégia compatível com as *Global Clock Networks* da FPGA MAX 10 da DE10-Lite.

Os seis estados de temporização e suas funções gerais estão resumidos na Tabela 3.1.

Tabela 3.1: Estados de temporização do SAP-1

Estado	Ciclo	Função
T1	Busca	Estado de Endereço — $PC \rightarrow MAR$
T2	Busca	Estado de Incremento — $PC = PC + 1$
T3	Busca	Estado de Memória — $RAM[MAR] \rightarrow IR$
T4	Execução	Depende do opcode (ver Seção 3.3)
T5	Execução	Depende do opcode (ver Seção 3.3)
T6	Execução	Depende do opcode (ver Seção 3.3)

3.2 Ciclo de Busca (T1, T2, T3)

Os três primeiros estados compõem o ciclo de busca (*fetch cycle*) e são idênticos para todas as instruções, independentemente do opcode. Sua função é localizar na memória a próxima instrução e carregá-la no Registrador de Instruções (IR).

3.2.1 T1 — Estado de Endereço

No estado T1, o Contador de Programa (PC) coloca seu endereço de 4 bits no Barramento W por meio do sinal **Ep** = 1 (Enable Program Counter). Simultaneamente, o MAR captura esse endereço com $\sim\mathbf{Lm} = 0$ (Load MAR). Dessa forma, o MAR fica apontando para a posição de memória onde a próxima instrução está armazenada.

```

1 if (ring[0]) begin    // T1
2     Ep    = 1'b1;      // PC -> Barramento W
3     Lm_n  = 1'b0;      // Barramento W -> MAR
4 end

```

Listing 3.2: Geração de sinais para o estado T1 em `controller_sequencer.v`.

3.2.2 T2 — Estado de Incremento

Com o endereço já salvo no MAR, o estado T2 incrementa o PC por meio do sinal **Cp** = 1 (Counter Program), preparando-o para apontar à próxima instrução.

```

1 else if (ring[1]) begin // T2
2     Cp = 1'b1;           // PC <- PC + 1
3 end

```

Listing 3.3: Geração de sinais para o estado T2 em `controller_sequencer.v`.

3.2.3 T3 — Estado de Memória

No estado T3, a RAM recebe o endereço armazenado no MAR e disponibiliza o byte correspondente no Barramento W por meio de $\sim\text{CE} = 0$ (Chip Enable da RAM). O Registrador de Instruções captura esse byte com $\sim\text{Li} = 0$ (Load IR). Ao final de T3, o ciclo de busca está concluído: o IR contém a instrução a ser executada, e o nibble mais significativo (opcode) já está disponível para o controlador.

```

1 else if (ring[2]) begin // T3
2     CE_n = 1'b0;         // RAM[MAR] -> Barramento W
3     Li_n = 1'b0;         // Barramento W -> IR
4 end

```

Listing 3.4: Geração de sinais para o estado T3 em `controller_sequencer.v`.

3.3 Ciclo de Execução (T4, T5, T6)

Os estados T4, T5 e T6 compõem o ciclo de execução (*execute cycle*). Ao contrário do ciclo de busca, os sinais de controle gerados nestes estados dependem do opcode presente no IR. O controlador implementa essa lógica com um bloco combinacional `case (opcode)` aninhado dentro de cada estado.

O SAP-1 suporta cinco instruções, cujos opcodes estão definidos como parâmetros locais no módulo `controller_sequencer.v`:

```

1 localparam LDA = 4'b0000; // 0x0
2 localparam ADD = 4'b0001; // 0x1
3 localparam SUB = 4'b0010; // 0x2
4 localparam OUT = 4'b1110; // 0xE
5 localparam HLT = 4'b1111; // 0xF

```

Listing 3.5: Definição dos opcodes das instruções em `controller_sequencer.v`.

3.3.1 Instrução LDA (Load Accumulator)

A instrução LDA carrega no Acumulador A o conteúdo da posição de memória cujo endereço está no campo operando da instrução.

- **T4:** O nibble baixo do IR (operando/endereço) é colocado no barramento via $\sim\mathbf{Ei} = 0$, e o MAR captura esse endereço ($\sim\mathbf{Lm} = 0$).
- **T5:** A RAM lê o endereço do MAR e disponibiliza o dado no barramento ($\sim\mathbf{CE} = 0$); o Acumulador carrega esse dado ($\sim\mathbf{La} = 0$).
- **T6:** Estado ocioso — nenhum sinal ativo.

3.3.2 Instrução ADD (Adição)

A instrução ADD soma o conteúdo de uma posição de memória ao Acumulador A e armazena o resultado no próprio Acumulador.

- **T4:** Igual ao LDA — operando do IR para o MAR ($\sim\mathbf{Ei} = 0$, $\sim\mathbf{Lm} = 0$).
- **T5:** RAM $\rightarrow$ barramento ($\sim\mathbf{CE} = 0$); Registrador B captura o dado ($\sim\mathbf{Lb} = 0$).
- **T6:** A ALU coloca o resultado da soma no barramento ($\mathbf{Eu} = 1$, $\mathbf{Su} = 0$) e o Acumulador armazena esse resultado ($\sim\mathbf{La} = 0$).

3.3.3 Instrução SUB (Subtração)

A instrução SUB subtrai o conteúdo de uma posição de memória do Acumulador A. Utiliza complemento de dois, controlado pelo sinal **Su**.

- **T4:** Idêntico ao ADD — operando do IR para o MAR.
- **T5:** RAM $\rightarrow$ barramento ($\sim\mathbf{CE} = 0$); Registrador B captura o dado ($\sim\mathbf{Lb} = 0$).
- **T6:** A ALU calcula $A - B$ com $\mathbf{Su} = 1$ e coloca o resultado no barramento ($\mathbf{Eu} = 1$); o Acumulador armazena o resultado ($\sim\mathbf{La} = 0$).

3.3.4 Instrução OUT (Saída)

A instrução OUT transfere o conteúdo do Acumulador A para o Registrador de Saída, que aciona os LEDs da DE10-Lite.

- **T4:** O Acumulador coloca seu valor no barramento (**Ea** = 1) e o Registrador de Saída captura esse valor ($\sim$ **Lo** = 0).
- **T5 e T6:** Estados ociosos.

3.3.5 Instrução HLT (Halt)

A instrução HLT ativa o sinal $hlt = 1$ a partir de T4, fazendo com que o *ring counter* deixe de avançar. Nenhum dado é transferido; o processador congela no estado atual até que um reset seja aplicado.

3.4 Tabela de Controle Completa

A Tabela 3.2 consolida a palavra de controle de 12 bits para cada estado e instrução do processador. Os sinais ativos em baixo são indicados com o prefixo “ $\sim$ ”. O valor “0” em um sinal ativo em alto (Cp, Ep, Ea, Su, Eu) significa inativo; o valor “1” em um sinal ativo em baixo ($\sim$ Lm, $\sim$ CE, $\sim$ Li, $\sim$ Ei, $\sim$ La, $\sim$ Lb, $\sim$ Lo) também significa inativo.

Tabela 3.2: Tabela completa de controle do SAP-1

Ciclo	Operação	Estado	Cp	Ep	$\sim$ Lm	$\sim$ CE	$\sim$ Li	$\sim$ Ei	$\sim$ La	Ea	Su	Eu	$\sim$ Lb	$\sim$ Lo
Busca	Todas	T1	0	1	0	1	1	1	1	0	0	0	1	1
Busca	Todas	T2	1	0	1	1	1	1	1	0	0	0	1	1
Busca	Todas	T3	0	0	1	0	0	1	1	0	0	0	1	1
Execução	LDA	T4	0	0	0	1	1	0	1	0	0	0	1	1
Execução	LDA	T5	0	0	1	0	1	1	0	0	0	0	1	1
Execução	LDA	T6	0	0	1	1	1	1	1	0	0	0	1	1
Execução	ADD	T4	0	0	0	1	1	0	1	0	0	0	1	1
Execução	ADD	T5	0	0	1	0	1	1	1	0	0	0	0	1
Execução	ADD	T6	0	0	1	1	1	1	0	0	0	1	1	1
Execução	SUB	T4	0	0	0	1	1	0	1	0	0	0	1	1
Execução	SUB	T5	0	0	1	0	1	1	1	0	0	0	0	1
Execução	SUB	T6	0	0	1	1	1	1	0	0	1	1	1	1
Execução	OUT	T4	0	0	1	1	1	1	1	1	0	0	1	0
Execução	OUT	T5	0	0	1	1	1	1	1	0	0	0	1	1
Execução	OUT	T6	0	0	1	1	1	1	1	0	0	0	1	1
Execução	HLT	T4-T6	(hlt = 1 — nenhum sinal de dados ativo)											

A **palavra de controle** tem 12 bits organizados na seguinte ordem, do mais significativo ao menos significativo:

$$\text{CON} = \underbrace{Cp}_{11} \underbrace{Ep}_{10} \underbrace{\overline{Lm}}_9 \underbrace{\overline{CE}}_8 \underbrace{\overline{Li}}_7 \underbrace{\overline{Ei}}_6 \underbrace{\overline{La}}_5 \underbrace{Ea}_4 \underbrace{Su}_3 \underbrace{Eu}_2 \underbrace{\overline{Lb}}_1 \underbrace{\overline{Lo}}_0$$

Observa-se que T4 é igual para LDA, ADD e SUB (todos necessitam buscar o operando na memória), diferindo apenas nos estados T5 e T6, onde a operação específica é realizada.

3.5 Diagrama da FSM

O diagrama da Figura 3.1 representa a Máquina de Estados Finitos do Processador de 8 Bits com todas as transições e sinais de controle ativos em cada estado. A FSM parte do estado T1 após o reset e percorre obrigatoriamente T1→T2→T3 (ciclo de busca) antes de ramificar nos estados T4–T6 conforme o opcode decodificado.

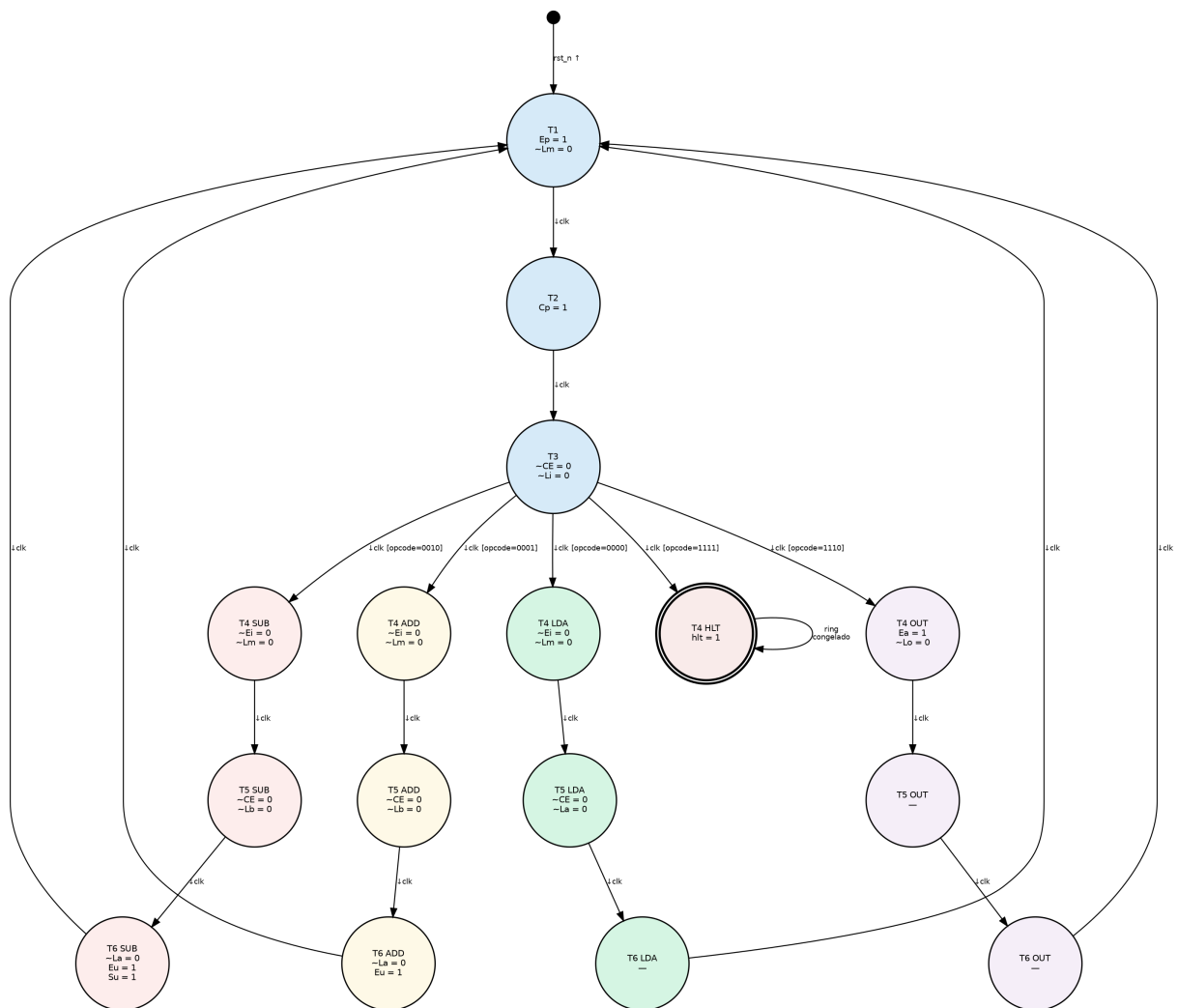


Figura 3.1: Diagrama da Máquina de Estados Finitos do Processador de 8 bits. Os estados T1–T3 compõem o ciclo de busca (idêntico para todas as instruções). Os estados T4–T6 ramificam-se conforme o opcode: LDA (0000), ADD (0001), SUB (0010), OUT (1110) e HLT (1111). O estado HLT é absorvente — o processador permanece nele até um reset.

Destacam-se as seguintes características do diagrama:

- **Reset assíncrono:** qualquer borda descendente em `rst_n` redireciona o fluxo imediatamente para T1.
- **Ciclo de busca unificado:** todos os cinco opcodes compartilham as mesmas transições $T1 \rightarrow T2 \rightarrow T3$, o que simplifica o hardware e reduz a lógica combinacional.
- **Bifurcação em $T3 \rightarrow T4$:** após a decodificação do opcode no IR, o controlador escolhe o ramo correto de execução. LDA, ADD e SUB seguem caminhos $T4 \rightarrow T5 \rightarrow T6$ distintos; OUT conclui em T4; HLT paralisa o processador.
- **Estados ociosos:** alguns estados não ativam sinal algum (por exemplo, T6 para LDA e OUT). Isso é previsto pela Lista 4, que permite estados sem operação quando a instrução não necessita de todos os seis ciclos.
- **Retorno automático a T1:** ao completar T6 (e quando `hlt = 0`), o *ring counter* rotaciona de volta para T1, iniciando o ciclo de busca da instrução seguinte.

A implementação Verilog valida esse comportamento: o bloco `always @(*)` do `controller_sequencer.v` é puramente combinacional, atualizado instantaneamente a cada mudança de `ring` ou `opcode`, garantindo que os sinais de controle estejam estabilizados antes da próxima borda ativa do clock — requisito fundamental para a operação síncrona correta do processador.

4.1.1 Interface com a DE10-Lite

As portas externas do módulo são mapeadas diretamente para os recursos de hardware da placa:

- **MAX10_CLK1_50** — clock de 50 MHz proveniente do pino PIN_P11 da DE10-Lite, utilizado como referência para o divisor de clock;
- **KEY[1:0]** — botões de pressão da placa. **KEY[0]** é conectado ao sinal de reset global (**rst_n**), ativo em nível baixo;
- **LEDR[9:0]** — LEDs vermelhos da placa. Os bits [7:0] exibem o resultado binário armazenado no registrador de saída; **LEDR[8]** acende quando a instrução **HLT** é executada; **LEDR[9]** pisca em compasso com o clock lento, funcionando como indicador de atividade (*heartbeat*).

4.1.2 Árbitro do Barramento W

A família MAX 10 não suporta lógica tristate interna. Por isso, o barramento W de 8 bits é implementado como um multiplexador prioritário no top-level, que seleciona qual módulo dirige o barramento a cada instante. A arbitragem é expressa pelo operador ternário encadeado abaixo, onde a prioridade vai da mais alta (PC) à mais baixa (ALU):

```

1 assign w_bus = Ep      ? pc_bus_out      :
2                      (!CE_n) ? ram_bus_out :
3                      (!Ei_n) ? ir_bus_out  :
4                      Ea      ? acum_bus_out :
5                      Eu      ? alu_bus_out  :
6                      8'h00;
```

Listing 4.1: Árbitro do Barramento W em `processador_8_bits_top.v`.

A palavra de controle gerada pelo controlador/sequenciador garante que, a cada estado de temporização, no máximo um desses sinais de habilitação esteja ativo, eliminando conflitos no barramento.

4.1.3 Instâncias e Conexões

O módulo instancia, na ordem: `clock_divider`, `program_counter`, `mar`, `ram_16x8`, `instruction_register`, `accumulator`, `register_b`, `alu`, `output_register` e `controller_sequencer`. Todos os submódulos recebem o clock lento (`clk_slow`) e o reset (`rst_n`), exceto o `clock_divider`, que opera com o clock de 50 MHz.

4.2 `clock_divider.v` — Divisor de Clock

O módulo `clock_divider` recebe o clock de 50 MHz da placa e gera um clock lento para que a execução do processador seja visível nos LEDs.

4.2.1 Interface

- **Entradas:** `clk_50` (50 MHz), `rst_n` (reset assíncrono, ativo em nível baixo);
- **Saídas:** `clk_out` (clock dividido);
- **Parâmetro:** `MAX_COUNT` (padrão: 25_000_000).

4.2.2 Funcionamento

O módulo mantém um contador interno de 25 bits que é incrementado a cada borda de subida do clock de 50 MHz. Quando o contador atinge o valor `MAX_COUNT - 1`, ele é zerado e o sinal `clk_out` é invertido. Com o parâmetro padrão de 25 000 000, a saída alterna a cada 25 milhões de ciclos de 50 MHz, gerando um clock de aproximadamente 1 Hz — frequência ideal para observação visual nos LEDs.

O parâmetro `MAX_COUNT` pode ser reduzido para valores menores durante a simulação no testbench, acelerando a execução sem necessidade de alterar o código de produção.

O reset é assíncrono: ao nível baixo em `rst_n`, o contador e a saída são zerados imediatamente, independentemente do clock.

4.3 `program_counter.v` — Contador de Programa

O módulo `program_counter` implementa o Contador de Programa (PC) de 4 bits, responsável por armazenar e fornecer o endereço da próxima instrução a ser buscada na RAM.

4.3.1 Interface

- **Entradas:** `clk`, `rst_n`, `Cp` (incremento), `Ep` (habilitação de saída);
- **Saídas:** `pc_out [3:0]` (valor interno do PC), `bus_out [7:0]` (saída para o barramento W).

4.3.2 Funcionamento

O PC é incrementado na borda de subida do clock quando o sinal Cp está em nível alto. O reset assíncrono (rst_n) zera o PC para o endereço 0000, fazendo o processador retomar a execução desde a primeira instrução armazenada na RAM.

A saída para o barramento é implementada como um mux: quando Ep = 1, os 4 bits do PC são colocados nos bits menos significativos de bus_out, com os 4 bits superiores zerados; quando Ep = 0, a saída retorna 0x00 (o árbitro do barramento no top-level ignora esse valor).

4.4 mar.v — Registrador de Endereço de Memória

O módulo mar implementa o Registrador de Endereço de Memória (MAR) de 4 bits, que armazena o endereço que será utilizado para acessar a RAM.

4.4.1 Interface

- **Entradas:** clk, rst_n, Lm_n (load, ativo em nível baixo), bus_in[7:0] (barramento W);
- **Saídas:** mar_addr[3:0] (endereço para a RAM).

4.4.2 Funcionamento

Na borda de subida do clock, quando Lm_n = 0, o MAR captura os 4 bits menos significativos do barramento W (bus_in[3:0]) e os armazena em mar_addr. O endereço capturado é mantido e enviado continuamente para a RAM até uma nova carga. O reset zera o registrador de forma assíncrona.

O MAR é utilizado em dois momentos distintos do ciclo de execução: em T1, para receber o endereço da próxima instrução vindo do PC; e em T4, para receber o operando da instrução (endereço do dado a ser acessado), proveniente do nibble inferior do IR.

4.5 ram_16x8.v — Memória RAM

O módulo ram_16x8 implementa a memória RAM de 16 posições de 8 bits cada, conforme especificado na Lista 4. Ela armazena tanto o código do programa quanto os dados utilizados durante a execução.

4.5.1 Interface

- **Entradas:** CE_n (chip enable, ativo em nível baixo), addr [3:0] (endereço de 4 bits vindo do MAR);
- **Saídas:** bus_out [7:0] (dado para o barramento W).

4.5.2 Programa de Teste Pré-carregado

A memória é inicializada com um programa de teste através de um bloco `initial`, que é sintetizável na MAX 10 como inicialização de RAM. O programa executa a seguinte sequência de operações:

Tabela 4.1: Conteúdo da RAM — programa de teste

Endereço	Binário	Instrução	Comentário
0x0	0000_1101	LDA 13	Acumulador $\leftarrow$ RAM[13] = 28
0x1	0001_1110	ADD 14	Acumulador $\leftarrow$ 28 + 14 = 42
0x2	0010_1111	SUB 15	Acumulador $\leftarrow$ 42 - 10 = 32
0x3	1110_0000	OUT	Saída $\leftarrow$ 32
0x4	1111_0000	HLT	Para o processamento
0xD	0001_1100	dado: 28	Primeiro operando
0xE	0000_1110	dado: 14	Segundo operando
0xF	0000_1010	dado: 10	Terceiro operando

O resultado esperado nos LEDs ao final da execução é 32 = 0010_0000 (em binário de 8 bits).

4.5.3 Funcionamento

A leitura é assíncrona: quando CE_n = 0, o dado armazenado no endereço indicado pelo MAR é imediatamente disponibilizado em bus_out. Quando CE_n = 1, a saída retorna 0x00, liberando o barramento. A RAM é somente de leitura durante a execução normal do processador.

4.6 instruction_register.v — Registrador de Instrução

O módulo `instruction_register` implementa o Registrador de Instruções (IR) de 8 bits, que recebe e armazena a instrução buscada na RAM.

4.6.1 Interface

- **Entradas:** `clk`, `rst_n`, `Li_n` (load, ativo em nível baixo), `Ei_n` (enable de saída, ativo em nível baixo), `bus_in[7:0]`;
- **Saídas:** `opcode[3:0]` (nibble superior, contínuo), `bus_out[7:0]` (nibble inferior para o barramento).

4.6.2 Funcionamento

Na borda de subida do clock, quando `Li_n = 0`, o IR captura os 8 bits do barramento W e os armazena internamente. O conteúdo armazenado é então dividido em dois nibbles:

- Os 4 bits superiores (`ir_reg[7:4]`) formam o opcode, que é enviado de forma contínua ao controlador/sequenciador para decodificação;
- Os 4 bits inferiores (`ir_reg[3:0]`) formam o operando (endereço de memória). Quando `Ei_n = 0`, esse nibble é colocado nos bits menos significativos do barramento W para que o MAR possa capturá-lo.

4.7 `accumulator.v` — Acumulador A

O módulo `accumulator` implementa o Acumulador A de 8 bits, registrador central do SAP-1 que armazena resultados intermediários durante a execução.

4.7.1 Interface

- **Entradas:** `clk`, `rst_n`, `La_n` (load, ativo em nível baixo), `Ea` (enable de saída, ativo em nível alto), `bus_in[7:0]`;
- **Saídas:** `acum_out[7:0]` (saída direta para a ALU, sempre ativa), `bus_out[7:0]` (saída para o barramento W).

4.7.2 Funcionamento

Conforme especificado na Lista 4, o acumulador possui duas saídas independentes:

- **Saída para a ALU** (`acum_out`): sempre ativa, alimenta continuamente o somador/subtrator com o valor atual do acumulador, independentemente dos sinais de controle;

- **Saída para o barramento W** (`bus_out`): habilitada somente quando $Ea = 1$, permitindo que o valor do acumulador seja transferido para o registrador de saída (instrução OUT) ou utilizado em outras operações.

A carga do registrador ocorre na borda de subida do clock quando $La_n = 0$. O reset assíncrono zera o acumulador.

4.8 `register_b.v` — Registrador B

O módulo `register_b` implementa o Registrador B de 8 bits, utilizado exclusivamente para fornecer o segundo operando às operações aritméticas da ALU.

4.8.1 Interface

- **Entradas:** `clk`, `rst_n`, `Lb_n` (load, ativo em nível baixo), `bus_in[7:0]`;
- **Saídas:** `b_out[7:0]` (saída direta para a ALU, sempre ativa).

4.8.2 Funcionamento

O registrador captura o dado presente no barramento W na borda de subida do clock quando $Lb_n = 0$. Diferentemente do acumulador, o Registrador B não possui saída para o barramento W — sua saída (`b_out`) alimenta diretamente e continuamente a ALU, sem necessidade de sinal de habilitação. O reset é assíncrono.

Este registrador é carregado durante o estado T5 das instruções ADD e SUB, quando o dado lido da RAM (o segundo operando) é transferido do barramento para o Registrador B.

4.9 `alu.v` — Somador/Subtrator

O módulo `alu` implementa a Unidade Lógico-Aritmética (ALU) simplificada do SAP-1, capaz de realizar adição e subtração de operandos de 8 bits.

4.9.1 Interface

- **Entradas:** `a_in[7:0]` (do Acumulador A), `b_in[7:0]` (do Registrador B), `Su` (seleção de operação: 0 = soma, 1 = subtração), `Eu` (enable de saída, ativo em nível alto);
- **Saídas:** `bus_out[7:0]` (resultado para o barramento W).

4.9.2 Funcionamento

A ALU opera de forma assíncrona: o resultado é calculado continuamente e sempre disponível internamente. Quando $Su = 0$, a saída é a soma $A + B$; quando $Su = 1$, a saída é a diferença $A - B$, implementada automaticamente pelo Verilog com base em complemento de dois.

O resultado só é colocado no barramento W quando $Eu = 1$. Isso ocorre no estado T6 das instruções ADD e SUB, momento em que o acumulador também é habilitado para carga ($La_n = 0$), fazendo o resultado da operação ser armazenado imediatamente no acumulador.

4.10 output_register.v — Registrador de Saída

O módulo `output_register` implementa o Registrador de Saída de 8 bits, responsável por capturar o resultado final do acumulador e mantê-lo estável para exibição nos LEDs da DE10-Lite.

4.10.1 Interface

- **Entradas:** `clk`, `rst_n`, `Lo_n` (load, ativo em nível baixo), `bus_in[7:0]`;
- **Saídas:** `out_data[7:0]` (saída contínua para os LEDs `LEDR[7:0]`).

4.10.2 Funcionamento

O registrador captura o dado do barramento W na borda de subida do clock quando $Lo_n = 0$. A instrução OUT ativa simultaneamente $Ea = 1$ (Acumulador coloca seu valor no barramento) e $Lo_n = 0$ (Registrador de Saída captura esse valor), completando a transferência em um único ciclo de clock.

O módulo não possui saída para o barramento W : seu único destino é o display binário formado pelos 8 LEDs vermelhos (`LEDR[7:0]`) da placa. O reset assíncrono apaga todos os LEDs.

4.11 controller_sequencer.v — Controlador/Sequenciador

O módulo `controller_sequencer` é o núcleo de controle do SAP-1. Ele implementa a Máquina de Estados Finitos (FSM) com seis estados de temporização (T1–T6), usando um ring

counter (contador circular), e gera a palavra de controle de 12 bits que comanda todos os demais módulos.

4.11.1 Interface

- **Entradas:** `clk`, `rst_n`, `opcode[3:0]` (vindo do IR);
- **Saídas:** os 12 sinais da palavra de controle — `Cp`, `Ep`, `Lm_n`, `CE_n`, `Li_n`, `Ei_n`, `La_n`, `Ea`, `Su`, `Eu`, `Lb_n`, `Lo_n` — e o sinal `hlt` (parada do processador).

4.11.2 Ring Counter

O sequenciamento dos estados é realizado por um registrador de deslocamento circular de 6 bits (ring), onde cada bit representa um estado (T1 a T6). Apenas um bit está em nível alto a cada instante. O ring counter é acionado pela borda negativa do clock, conforme especificado na Lista 4, para que os sinais de controle fiquem estáveis antes da borda de subida na qual os registradores capturam os dados.

Após o reset, o estado inicial é T1 (ring = 6'b000001). A cada borda negativa do clock, o bit alto avança: T1→T2→T3→T4→T5→T6→T1→... Quando `hlt` = 1, a rotação é suspensa e o processador congela no estado em que se encontrava.

4.11.3 Geração da Palavra de Controle

A lógica combinacional (bloco `always @(*)`) lê o estado atual do ring counter e o opcode da instrução em execução para determinar quais sinais de controle devem ser ativados. O valor padrão (inativo) de cada sinal é:

- Sinais ativos em nível baixo (`Lm_n`, `CE_n`, `Li_n`, `Ei_n`, `La_n`, `Lb_n`, `Lo_n`): valor inativo = 1;
- Sinais ativos em nível alto (`Cp`, `Ep`, `Ea`, `Su`, `Eu`, `hlt`): valor inativo = 0.

O **ciclo de busca** (T1, T2, T3) é idêntico para todas as instruções:

- **T1** — `Ep` = 1, `Lm_n` = 0: o endereço do PC é transferido para o MAR;
- **T2** — `Cp` = 1: o PC é incrementado;
- **T3** — `CE_n` = 0, `Li_n` = 0: a instrução lida da RAM é carregada no IR.

O ciclo de execução (T4, T5, T6) depende do opcode decodificado por uma estrutura case:

- **LDA**: em T4, o operando do IR é enviado ao MAR; em T5, o dado da RAM é carregado no acumulador; T6 é ocioso;
- **ADD**: em T4, o operando do IR vai ao MAR; em T5, o dado da RAM é carregado no Registrador B; em T6, a ALU soma $A + B$ e o resultado é armazenado no acumulador;
- **SUB**: idêntico ao ADD em T4 e T5; em T6, ativa-se adicionalmente $S_u = 1$ para que a ALU realize a subtração $A - B$;
- **OUT**: em T4, $E_a = 1$ e $Lo_n = 0$ transferem o acumulador para o registrador de saída; T5 e T6 são ociosos;
- **HLT**: $hlt = 1$ é ativado em T4 e mantido nos estados seguintes, suspendendo o ring counter e parando o processador.

Capítulo 5

Programa de Teste

5.1 Descrição do Programa Carregado na RAM

O programa de teste foi elaborado para exercitar as cinco instruções do conjunto de instruções do SAP-1 — LDA, ADD, SUB, OUT e HLT — e produzir um resultado numericamente verificável nos LEDs da placa DE10-Lite. O conteúdo da memória RAM 16×8 está pré-carregado diretamente no bloco `initial` do módulo `ram_16x8.v`, conforme a Tabela 5.1.

Tabela 5.1: Conteúdo da RAM 16×8 após inicialização

Endereço (hex)	Binário (8 bits)	Instrução / Dado	Comentário
0x0	0000_1101	LDA 13	Carrega RAM[13]=28 no acumulador
0x1	0001_1110	ADD 14	Acumulador $\leftarrow 28 + \text{RAM}[14] = 42$
0x2	0010_1111	SUB 15	Acumulador $\leftarrow 42 - \text{RAM}[15] = 32$
0x3	1110_0000	OUT	Registrador de saída $\leftarrow 32$
0x4	1111_0000	HLT	Para o processamento
0x5–0xC	0000_0000	—	Posições não utilizadas
0xD	0001_1100	Dado: 28	Primeiro operando (0x1C)
0xE	0000_1110	Dado: 14	Segundo operando (0x0E)
0xF	0000_1010	Dado: 10	Terceiro operando (0x0A)

O formato de cada instrução segue o padrão definido na Lista 4: os quatro bits mais significativos formam o opcode e os quatro bits menos significativos formam o operando (campo de endereço). Assim, a instrução 0000_1101 corresponde ao opcode LDA (0000) com operando 13 (1101), indicando que o acumulador deve ser carregado com o conteúdo do endereço de memória 0xD.

5.2 Execução Passo a Passo (Trace Completo)

Para cada instrução, o controlador-sequenciador percorre seis estados de temporização (T1–T6), implementados por um contador circular (*ring counter*) acionado na borda negativa do clock. Os três primeiros estados (T1–T3) compõem o ciclo de busca, idêntico para todas as instruções, e os três últimos (T4–T6) compõem o ciclo de execução, dependente do opcode.

A seguir é apresentado o trace completo da execução do programa.

Instrução 1 — LDA 13 (endereço 0x0)

T1 (Endereço): $E_p = 1$, $\overline{L_m} = 0$. O valor atual do Program Counter (PC = 0x0) é colocado no barramento W; o MAR captura o endereço 0x0 na próxima borda de subida do clock.

T2 (Incremento): $C_p = 1$. O PC é incrementado para 0x1.

T3 (Memória): $\overline{CE} = 0$, $\overline{L_i} = 0$. A RAM coloca o conteúdo do endereço 0x0 (0000_1101) no barramento W; o Registrador de Instruções (IR) captura essa palavra. O opcode 0000 (LDA) é enviado ao controlador.

T4: $\overline{E_i} = 0$, $\overline{L_m} = 0$. O operando do IR (0xD = 13) é colocado no barramento W; o MAR captura o endereço 0xD.

T5: $\overline{CE} = 0$, $\overline{L_a} = 0$. A RAM coloca o dado do endereço 0xD (valor 28 = 0001_1100) no barramento W; o acumulador A captura 28.

T6 (ocioso): Nenhum sinal de controle ativo.

Estado do acumulador: A = 28.

Instrução 2 — ADD 14 (endereço 0x1)

T1–T3 (busca): Igual ao ciclo anterior; o IR captura 0001_1110 (opcode ADD, operando 0xE = 14).

T4: $\overline{E_i} = 0$, $\overline{L_m} = 0$. O operando 0xE vai ao barramento W; o MAR captura o endereço 0xE.

T5: $\overline{CE} = 0$, $\overline{L_b} = 0$. A RAM coloca o dado do endereço 0xE (valor 14 = 0000_1110) no barramento W; o Registrador B captura 14.

T6: $\overline{L_a} = 0$, $E_u = 1$, $S_u = 0$ (adição). A ALU calcula $28 + 14 = 42$ e coloca o resultado no barramento W; o acumulador A captura 42.

Estado do acumulador: A = 42.

Instrução 3 — SUB 15 (endereço 0x2)

T1–T3 (busca): O IR captura 0010_1111 (opcode SUB, operando 0xF = 15).

T4: $\overline{E_i} = 0$, $\overline{L_m} = 0$. O operando 0xF vai ao barramento W; o MAR captura o endereço 0xF.

T5: $\overline{CE} = 0$, $\overline{L_b} = 0$. A RAM coloca o dado do endereço 0xF (valor 10 = 0000_1010) no barramento W; o Registrador B captura 10.

T6: $\overline{L_a} = 0$, $E_u = 1$, $S_u = 1$ (subtração por complemento de dois). A ALU calcula $42 - 10 = 32$ e coloca o resultado no barramento W; o acumulador A captura 32.

Estado do acumulador: A = 32.

Instrução 4 — OUT (endereço 0x3)

T1–T3 (busca): O IR captura 1110_0000 (opcode OUT).

T4: $E_a = 1$, $\overline{L_o} = 0$. O valor do acumulador (32) é colocado no barramento W pelo sinal E_a ; o Registrador de Saída captura 32 na borda de subida do clock. O valor 32 (0010_0000) passa a ser exibido nos LEDs LEDR[7:0] da DE10-Lite.

T5–T6 (ociosos): Nenhum sinal de controle ativo.

Instrução 5 — HLT (endereço 0x4)

T1–T3 (busca): O IR captura 1111_0000 (opcode HLT).

T4: O sinal `hlt` é elevado a 1 pelo controlador. O ring counter congela — não avança mais — e o processador para de executar novas instruções. O LED LEDR[8] acende, sinalizando o estado de halt.

5.3 Resultado Esperado

Ao término do programa, o Registrador de Saída deve conter o valor $28 + 14 - 10 = 32$, que em binário corresponde a 0010_0000. Nos LEDs da DE10-Lite, somente LEDR[5] deve estar aceso (bit de peso $2^5 = 32$), com os demais apagados. O LED LEDR[8] deve estar aceso, indicando que a instrução HLT foi executada com sucesso e o processador está parado.

Capítulo 6

Simulação e Testbench

6.1 Descrição dos Testbenches

Para validar o projeto antes da síntese no FPGA, foram desenvolvidos seis testbenches independentes no ModelSim. Cinco são testes unitários — cada um dedicado a um submódulo específico — e um é o teste de integração global do processador completo. Todos utilizam a diretiva ``timescale 1ns/1ps` e incluem um bloco `$dumpvars` para geração de arquivos VCD de formas de onda.

6.1.1 `tb_alu.v`

O testbench da ALU valida o somador/subtrator de 8 bits, que opera de forma assíncrona, sem clock. Os estímulos são organizados em quatro blocos: (1) isolamento do barramento com $E_u = 0$, verificando que `bus_out` permanece em 0x00 independentemente das entradas; (2) adições com $S_u = 0$ e $E_u = 1$; (3) subtrações via complemento de dois com $S_u = 1$ e $E_u = 1$; e (4) simulação completa do programa real ($28 + 14 = 42$; $42 - 10 = 32$). O testbench inclui verificação automática com contagem de erros armazenada na variável `erros`.

6.1.2 `tb_clock_divider.v`

O testbench do divisor de clock instancia o módulo com o parâmetro reduzido `MAX_COUNT = 5` para tornar a simulação viável sem aguardar os 25 milhões de ciclos da versão de hardware. São verificados o comportamento do reset assíncrono (saída deve permanecer em zero enquanto $\overline{rst} = 0$), a frequência de inversão do clock de saída (período esperado de 200 ns para `MAX_COUNT = 5` com clock de entrada de 20 ns) e o reset aplicado durante a operação normal.

6.1.3 `tb_program_counter.v`

O testbench do Program Counter valida o reset assíncrono, a inatividade dos sinais C_p e E_p , o incremento síncrono controlado por C_p , a saída para o barramento controlada por E_p , o overflow circular de 4'hF para 4'h0 e o reset no meio da contagem.

6.1.4 `tb_registradores.v`

Este testbench agrupa em um único arquivo os testes unitários de cinco módulos: Acumulador A, Registrador B, MAR, Registrador de Instruções (IR) e Registrador de Saída. Para cada módulo são verificados o reset assíncrono, a não-captura com sinal de load inativo, a captura correta na borda de subida com o sinal de load ativo, a persistência do dado após a desativação do load e o comportamento da saída para o barramento (quando aplicável). O IR recebe atenção especial para confirmar a separação correta entre nibble alto (opcode) e nibble baixo (operando), incluindo as instruções do programa real: LDA 13, ADD 14 e HLT.

6.1.5 `tb_controller_sequencer.v`

O testbench do controlador-sequenciador é o mais extenso entre os unitários. Ele verifica sistematicamente: (1) o estado T1 após o reset; (2) a progressão correta do ring counter em $T1 \rightarrow T2 \rightarrow T3$ pelos sinais de busca; (3) a palavra de controle gerada em cada estado T4, T5 e T6 para cada uma das quatro instruções LDA, ADD, SUB e OUT; e (4) o congelamento do ring counter com `hlt = 1` após a instrução HLT. Cada verificação é realizada pela tarefa `check_signal`, que compara o valor obtido com o esperado e contabiliza falhas.

6.1.6 `tb_processador_8_bits.v`

O testbench global instancia o top-level `processador_8_bits_top` com o parâmetro `MAX_COUNT = 2`, tornando o clock do processador suficientemente rápido para simulação. Após a aplicação de um pulso de reset de 40 ns, o testbench aguarda a detecção do sinal `LEDR[8] = 1` (halt) com um mecanismo de timeout para proteção contra travamentos. Uma vez detectado o halt, verifica se `LEDR[7:0] == 32` e reporta sucesso ou falha.

6.2 Resultados da Simulação

6.2.1 ALU — `tb_alu.v`

A Figura 6.1 apresenta as formas de onda obtidas no ModelSim para o testbench da ALU.

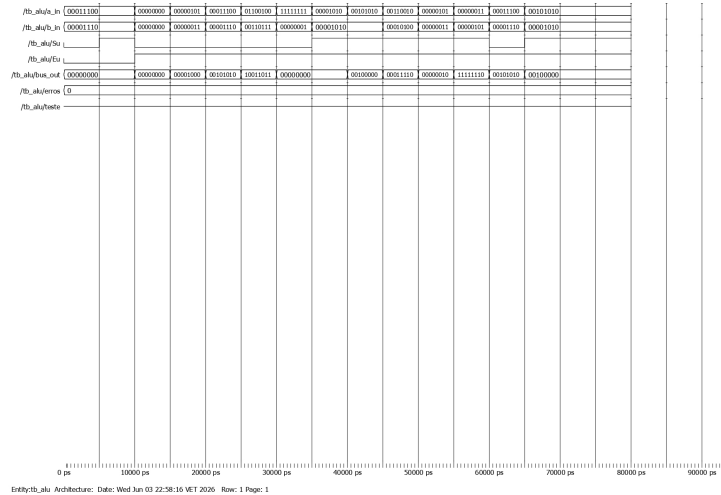


Figura 6.1: Waveform do testbench da ALU (`tb_alu`)

Na simulação, observa-se que `bus_out` permanece em 00000000 sempre que $E_u = 0$, confirmando o isolamento correto do barramento. Quando $E_u = 1$ e $S_u = 0$, o sinal `bus_out` reflete imediatamente a soma de `a_in` e `b_in`, demonstrando a natureza assíncrona do módulo — nenhuma borda de clock é necessária para atualizar o resultado. Quando $S_u = 1$, a saída corresponde à subtração calculada por complemento de dois, conforme especificado na Lista 4. As operações críticas do programa real ($28 + 14 = 42$ e $42 - 10 = 32$) foram verificadas explicitamente, com `bus_out` = 00101010 (42) e `bus_out` = 00100000 (32) obtidos corretamente. A variável `erros` permaneceu em zero ao término da simulação.

6.2.2 Divisor de Clock — `tb_clock_divider.v`

A Figura 6.2 apresenta as formas de onda do testbench do divisor de clock.

$C_p = 0$, o contador permanece estacionário. Quando $C_p = 1$ é habilitado, o contador incrementa a cada borda de subida do clock, passando de 0000 até 0100 e além, como pode ser visto nas transições no waveform. A saída bus_out reproduz os 4 bits do PC nos bits menos significativos apenas quando $E_p = 1$, permanecendo em 00000000 nos demais instantes. O reset assíncrono aplicado no meio da contagem zera pc_out imediatamente, sem aguardar a próxima borda de clock. A variável erros permaneceu em zero.

6.2.4 Registradores — tb_registradores.v

A Figura 6.4 apresenta as formas de onda do testbench conjunto dos registradores.

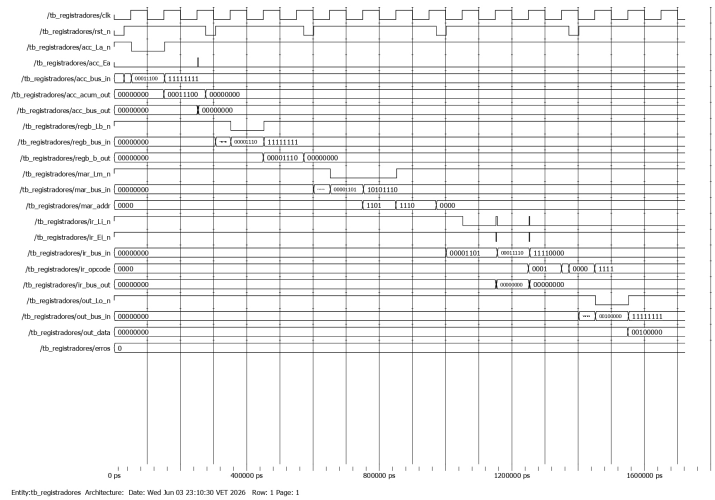


Figura 6.4: Waveform do testbench dos registradores (tb_registradores)

O waveform exhibe, em sequência, os sinais de todos os cinco módulos testados. Para o Acumulador, observa-se que a saída acum_out (disponível continuamente para a ALU) permanece com o valor capturado independentemente do sinal E_a , enquanto bus_out é condicionado exclusivamente a $E_a = 1$. Para o Registrador B, confirma-se a ausência de saída para o barramento (sem sinal Enable), com b_out alimentando a ALU diretamente. O MAR demonstra a captura correta apenas dos 4 bits menos significativos do barramento. O IR apresenta a separação correta entre nibble alto (opcode) e nibble baixo (operando), com o opcode 1111 (HLT) sendo reconhecido ao final. O Registrador de Saída captura o valor 32 (00100000) quando $\overline{L_o} = 0$, mantendo-o estável mesmo após $\overline{L_o}$ retornar a 1. A variável erros permaneceu em zero para todos os módulos.

6.2.5 Controlador-Sequenciador — tb_controller_sequencer.v

A Figura 6.5 apresenta as formas de onda do testbench do controlador-sequenciador.

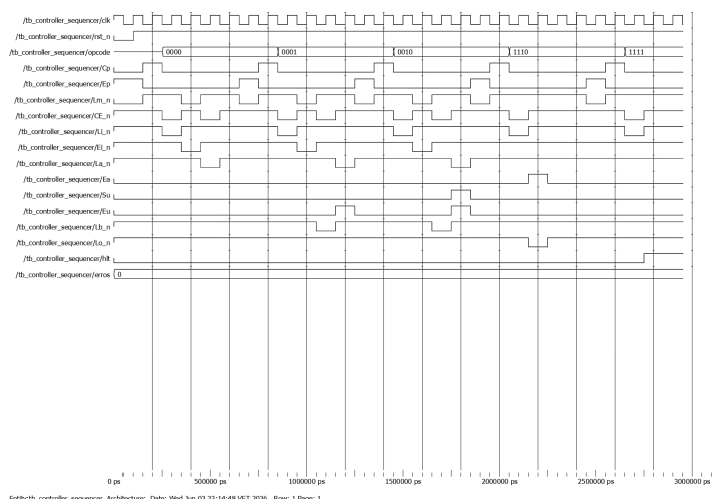


Figura 6.5: Waveform do testbench do controlador-sequenciador (tb_controller_sequencer)

O waveform confirma a progressão correta do ring counter pela sequência $T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow T5 \rightarrow T6 \rightarrow T1$, com cada transição ocorrendo na borda negativa do clock, conforme especificado na Lista 4. O campo opcode alterna entre os valores 0000 (LDA), 0001 (ADD), 0010 (SUB), 1110 (OUT) e 1111 (HLT), e em cada caso a palavra de controle gerada em T4, T5 e T6 corresponde exatamente à tabela de controle da Lista 4. Em particular, para SUB no estado T6, observa-se a ativação simultânea de $\overline{L_a}$, E_u e S_u , que é o ponto de maior distinção entre ADD e SUB. Para HLT, o sinal `hlt` eleva-se a 1 em T4 e o ring counter deixa de avançar, permanecendo congelado no estado de halt durante os ciclos seguintes. A variável `erros` permaneceu em zero.

6.2.6 Processador Completo — tb_processador_8_bits.v

A Figura 6.6 apresenta as formas de onda do testbench global do processador.

Na simulação global, o clock de 50 MHz é dividido pelo parâmetro `MAX_COUNT = 2`, gerando o clock lento do processador com período de 80 ns. Após o pulso de reset, o processador inicia a execução do programa. O waveform mostra o sinal `KEY` fixo em 11 (botões soltos, reset inativo) e os 10 bits do barramento `LEDR` sendo monitorados. O sinal `LEDR[8]` (`HLT`) permanece em nível baixo durante toda a execução das instruções `LDA`, `ADD`, `SUB` e `OUT`, e eleva-se a 1 após a busca e decodificação da instrução `HLT`, encerrando a simulação antes do timeout configurado. O sinal `LEDR[9]` (`heartbeat`) oscila em sincronia com o clock lento ao longo de toda a execução. Ao término, `LEDR[7:0]` apresenta o valor 0100100000 — onde os bits [7:0] correspondem a 00100000 = 32 decimal — confirmando o resultado correto do programa.

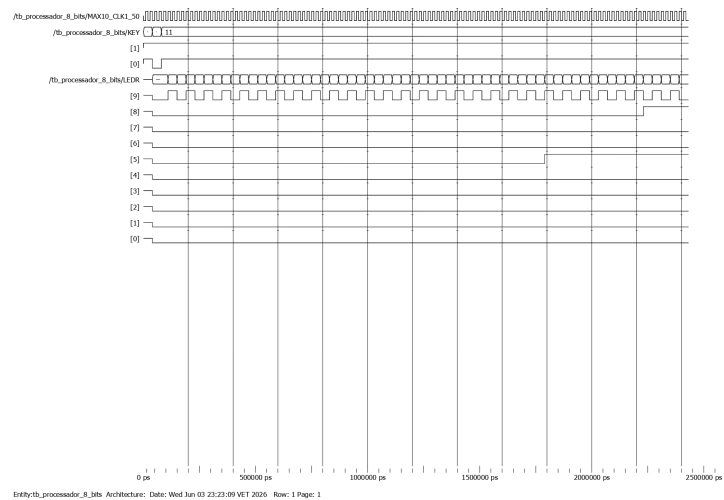


Figura 6.6: Waveform do testbench global (tb_processador_8_bits)

6.3 Verificação do Resultado

A Tabela 6.1 consolida os resultados obtidos em todas as simulações realizadas no ModelSim.

Tabela 6.1: Resumo da verificação por testbench

Testbench	Erros detectados	Status
tb_alu	0	SUCESSO
tb_clock_divider	0	SUCESSO
tb_program_counter	0	SUCESSO
tb_registradores	0	SUCESSO
tb_controller_sequencer	0	SUCESSO
tb_processador_8_bits	0	SUCESSO

Todos os testbenches reportaram zero erros. O resultado mais importante é o do testbench global: o processador SAP-1 executou corretamente as cinco instruções do programa de teste — LDA, ADD, SUB, OUT e HLT — produzindo nos LEDs o valor $28 + 14 - 10 = 32 = 0010_0000_2$, exatamente o resultado esperado conforme definido na Seção 5.3. A cadeia de validação, que vai dos testes unitários de cada submódulo até a integração completa, demonstra que a implementação em Verilog do processador SAP-1 está de acordo com a arquitetura e o conjunto de instruções descritos na Lista 4.

Capítulo 7

Implementação na FPGA DE10-Lite

A placa DE10-Lite, baseada no FPGA Intel MAX 10 (10M50DAF484C7G), foi utilizada como plataforma de síntese e verificação do processador. Esta seção descreve o mapeamento dos sinais do projeto para os recursos físicos disponíveis na placa, o projeto do divisor de clock e a interface de saída com os LEDs.

7.1 Mapeamento de Pinos

O módulo top-level do processador, `processador_8_bits_top`, é responsável por expor os sinais internos do circuito para os pinos físicos da DE10-Lite. O mapeamento foi definido no arquivo de restrições de pinos (`.qsf`) do Quartus Prime e corresponde aos recursos padrão da placa conforme seu manual de referência.

A Tabela 7.1 apresenta os sinais mapeados, seus pinos correspondentes e suas funções no contexto do projeto.

Tabela 7.1: Mapeamento de sinais para pinos da DE10-Lite

Sinal	Pino	Recurso	Função
MAX10_CLK1_50	PIN_P11	Clock 50 MHz	Clock principal do sistema
KEY[0]	PIN_B8	Push-button	Reset assíncrono (ativo em baixo)
LEDR[7:0]	PIN_A8–B11	LEDs vermelhos	Exibição do resultado em binário
LEDR[8]	PIN_A11	LED vermelho	Indicador de HLT (parado)
LEDR[9]	PIN_B12	LED vermelho	Heartbeat (pisca no ritmo do clock lento)

O sinal de reset `rst_n` é derivado diretamente de `KEY[0]`. Por ser ativo em nível baixo, o processador opera normalmente enquanto o botão não é pressionado. Ao pressionar `KEY[0]`, todos os registradores são zerados de forma assíncrona e o *Program Counter* retorna ao endereço zero, reiniciando a execução do programa.

7.2 Clock e Divisor de Frequência

A DE10-Lite fornece um clock de referência de 50 MHz no pino PIN_P11. Essa frequência é muito elevada para observação visual do funcionamento do processador nos LEDs, sendo necessário reduzi-la a aproximadamente 1 Hz.

Para isso, o projeto inclui o módulo `clock_divider`, que implementa um divisor de frequência por contagem. O módulo conta pulsos do clock de 50 MHz até atingir o valor configurado pelo parâmetro `MAX_COUNT` e, ao atingi-lo, inverte o sinal de saída e reinicia o contador. Isso gera um clock de saída com período igual ao dobro do tempo de contagem.

O cálculo da frequência de saída é dado por:

$$f_{out} = \frac{f_{in}}{2 \times MAX_COUNT} \quad (7.1)$$

$$f_{out} = \frac{50.000.000}{2 \times 25.000.000} = 1 \text{ Hz} \quad (7.2)$$

Para simulação, o parâmetro pode ser reduzido para `MAX_COUNT = 2`, acelerando a execução sem alterar a lógica do circuito. Esse valor é passado pelo módulo top-level via parâmetro genérico, mantendo o divisor reutilizável, conforme mostrado no Código 7.1.

```

1 clock_divider #(
2     .MAX_COUNT (25_000_000)
3 ) u_clk_div (
4     .clk_50 (MAX10_CLK1_50),
5     .rst_n  (rst_n),
6     .clk_out (clk_slow)
7 );

```

Listing 7.1: Instanciação do módulo `clock_divider`

O módulo utiliza um contador interno de 25 bits, suficiente para representar valores até 33.554.432, cobrindo o `MAX_COUNT` de 25.000.000 com folga. O reset é assíncrono e ativo em nível baixo, alinhado com o padrão do restante do projeto.

7.3 LEDs e Display Binário

A saída do processador é exibida nos dez LEDs vermelhos (`LEDR[9:0]`) disponíveis na DE10-Lite. Cada LED corresponde a um bit, com o LED de índice zero representando o bit menos significativo. O mapeamento adotado é o seguinte:

- **LEDR[7:0]** — exibe o conteúdo do Registrador de Saída em formato binário natural. Após a instrução OUT ser executada, o valor do Acumulador é capturado pelo Registrador de Saída e mantido nos LEDs até o próximo OUT ou reset.
- **LEDR[8]** — acende quando a instrução HLT é executada, indicando que o processador está parado. Enquanto HLT estiver ativo, o *ring counter* não avança e os demais sinais de controle permanecem congelados.
- **LEDR[9]** — conectado diretamente ao clock lento gerado pelo divisor. Pisca a aproximadamente 1 Hz, servindo como indicador visual de que o processador está em operação (*heartbeat*).

O mapeamento no top-level é direto, sem lógica adicional, conforme o Código 7.2.

```

1 assign LEDR[7:0] = out_data; // resultado do Registrador de Saida
2 assign LEDR[8]   = hlt;      // indicador de HLT
3 assign LEDR[9]   = clk_slow; // heartbeat

```

Listing 7.2: Atribuição dos sinais de saída aos LEDs

Como a MAX 10 não possui conversor digital-analógico nem display de sete segmentos conectado diretamente ao processador neste projeto, a leitura do resultado é feita interpretando os oito LEDs como um número binário de 8 bits. Por exemplo, o resultado 32 (decimal) corresponde ao padrão 0010 0000 nos LEDs, com apenas LEDR[5] aceso, conforme ilustrado na Figura 7.1.

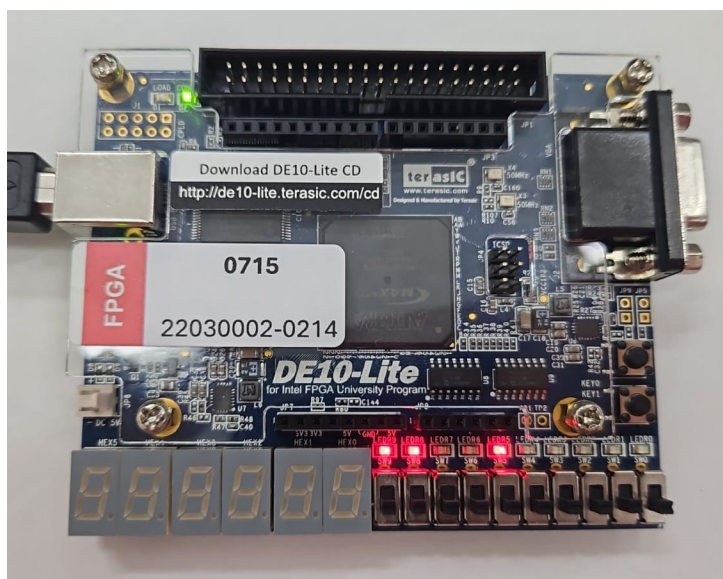


Figura 7.1: Resultado da execução na FPGA DE10-Lite exibindo o valor decimal 32 (binário 0010 0000) nos LEDs.

7.4 Extensão Proposta: Controle do Processador por um PC Hospedeiro

Uma questão levantada para reflexão é: *se este processador fosse conectado a um computador hospedeiro (host PC), que passasse a carregar os programas na memória e controlar sua execução externamente — em vez de utilizar o programa fixo pré-carregado no bloco `initial` da RAM — como essa interface deveria ser projetada?*

Na implementação atual, o programa executado pelo processador é fixo, carregado estaticamente na RAM 16×8 através do bloco `initial` do módulo `ram_16x8.v`, e o clock de execução é gerado internamente pelo `clock_divider`. Para que um PC hospedeiro pudesse carregar programas dinamicamente e controlar a execução, seriam necessárias três mudanças principais na arquitetura:

- a) **Canal de comunicação (UART).** A FPGA precisaria de um módulo UART (transmissor/receptor serial), conectado às portas RX/TX da DE10-Lite, para trocar bytes com o PC via porta serial (ou USB-serial). O PC enviaria comandos simples — por exemplo, um opcode de “escrever memória” seguido do endereço e do dado — e a FPGA responderia com status ou com os dados lidos;
- b) **Substituição da RAM estática por memória endereçável externamente.** A RAM 16×8 deixaria de ser inicializada por um bloco `initial` fixo e passaria a ter uma porta de escrita adicional controlada pelo módulo UART: quando o PC enviasse um comando de escrita, o receptor UART geraria os sinais `we` (*write enable*), `addr` e `data_in` para gravar cada instrução na posição correta antes do início da execução — um mecanismo equivalente ao *bootloader* de um microcontrolador;
- c) **Controle externo do ring counter (*start/stop/step*).** Em vez de o `clock_divider` gerar continuamente um clock livre, o PC controlaria a execução por meio de sinais de comando: um pulso de *step* avançaria o *ring counter* em um único estado de temporização (T1→T2 etc.), permitindo depuração ciclo a ciclo; um comando de *run* liberaria o clock lento normalmente; e um comando de *reset* reaplicaria `rst_n`. Isso poderia ser implementado com uma pequena FSM de controle intermediária entre a UART e o `controller_sequencer`, multiplexando a fonte do clock entre o `clock_divider` interno e os pulsos vindos do PC.

Do lado do software, o PC executaria um programa simples (por exemplo, em Python com a biblioteca `pyserial`) responsável por montar o programa em linguagem de máquina do SAP-1 (assemblar mnemônicos como LDA, ADD, SUB, OUT e HLT em opcodes de 8 bits), enviá-los byte a byte pela UART, disparar a execução e ler de volta o conteúdo do Registrador de Saída (`LEDR[7:0]`) para exibir o resultado no terminal do computador.

Essa extensão transformaria o SAP-1 de um circuito com programa fixo em um sistema mínimo *host-target*, no qual a FPGA atua como uma CPU escrava programável remotamente — um passo conceitual em direção à forma como processadores reais são carregados e depurados (por exemplo, via JTAG ou *bootloaders* seriais).

Capítulo 8

Conclusão

Este trabalho apresentou a implementação completa do processador educacional SAP-1 (Simple As Possible 1) em linguagem Verilog HDL, abrangendo desde a especificação da arquitetura em nível RTL até a síntese e verificação física na placa FPGA Terasic DE10-Lite. Os objetivos definidos inicialmente foram integralmente alcançados.

A arquitetura de barramento único de 8 bits foi implementada com sucesso por meio de um multiplexador prioritário, solução adotada para contornar a ausência de suporte a lógica tristate interna na família Intel MAX 10 – um desafio de projeto não previsto na arquitetura original de Malvino e Brown, mas resolvido sem comprometer a fidelidade ao comportamento especificado. A unidade de controle, projetada como uma máquina de estados finitos com seis estados de temporização (T1–T6) sequenciados por um contador em anel sensível à borda negativa do clock, mostrou-se capaz de gerar corretamente a palavra de controle de 12 bits para cada uma das cinco instruções do conjunto implementado: LDA, ADD, SUB, OUT e HLT.

A validação do projeto foi conduzida em duas etapas complementares. Na simulação, seis testbenches – cinco unitários e um de integração – confirmaram, sem qualquer erro detectado, o comportamento correto de cada submódulo isoladamente e do processador como um todo, incluindo o tratamento do reset assíncrono, a arbitragem do barramento e a execução completa do programa de teste. Na síntese física, o processador foi embarcado na DE10-Lite com o clock reduzido a aproximadamente 1 Hz por meio do módulo `clock_divider`, permitindo a observação visual, ciclo a ciclo, da execução do programa diretamente nos LEDs da placa. O resultado obtido – o valor decimal 32, correspondente à operação $28 + 14 - 10$ – foi confirmado tanto em simulação quanto na demonstração em hardware, evidenciando a consistência entre o modelo simulado e o circuito efetivamente sintetizado.

De um ponto de vista pedagógico, o desenvolvimento deste projeto permitiu consolidar, na prática, conceitos fundamentais de arquitetura de computadores frequentemente abordados apenas de forma teórica, como o ciclo de busca e execução de instruções, a geração de sinais de controle por uma unidade sequencial e as limitações impostas por tecnologias de hardware

específicas sobre decisões de projeto – a exemplo da arbitragem de barramento adotada em substituição ao tristate.

Como possíveis extensões deste trabalho, destacam-se: a ampliação do conjunto de instruções do SAP-1 (por exemplo, com operações de salto condicional, como no SAP-2), a substituição do display binário nos LEDs por um display de sete segmentos para leitura direta em decimal ou hexadecimal, e a adição de uma interface serial para carregamento dinâmico de programas na RAM, eliminando a necessidade de recompilação do bitstream a cada alteração do código-fonte executado.

Referências

- [1] MALVINO, Albert Paul; BROWN, Jerald A. **Digital Computer Electronics**. 3. ed. Nova York: McGraw-Hill, 1992.
- [2] TOCCI, Ronald J.; WIDMER, Neal S.; MOSS, Gregory L. **Sistemas Digitais: Princípios e Aplicações**. 11. ed. São Paulo: Pearson Prentice Hall, 2011.
- [3] PATTERSON, David A.; HENNESSY, John L. **Computer Organization and Design: The Hardware/Software Interface**. 5. ed. Waltham: Morgan Kaufmann, 2014.
- [4] Terasic Inc. **DE10-Lite User Manual**. Hsinchu: Terasic Technologies, 2016.
Disponível em: <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=1021>.
Acesso em: maio de 2026.